CYLIXLAB · STUDY GUIDE

Module 05

Generative AI, Large Language Models & AI Systems

Theory | LLM Foundations · Prompt Engineering · RAG · Fine-Tuning (LoRA) · Deployment · Safety & Ethics

What you will master

Section	Topic	You should be able to…
1	LLM Foundations	Explain how an LLM is trained (pre-training → SFT → preference tuning), count tokens and estimate cost and context budgets
2	Prompt Engineering	Structure a prompt, choose zero-shot, few-shot, chain-of-thought or structured output, and use tools
3	RAG Systems	Design a retrieval-augmented pipeline: chunking, embeddings, vector search, hybrid search, reranking and evaluation
4	Fine-Tuning and LoRA	Decide when to fine-tune, compute LoRA parameters and memory, and explain QLoRA and preference tuning
5	Production Deployment	Size GPU memory and KV cache, apply quantization and batching, measure latency and cost, and build reliable services
6	AI Safety and Ethics	Identify risks (hallucination, injection, bias, privacy), apply defence in depth and compute simple fairness metrics
7	Revision Kit	Choose between prompting, RAG and fine-tuning, use the formula sheet and avoid classic mistakes

KEY IDEA

The big idea: a large language model is a Transformer trained to predict the next token, then shaped into an assistant. Building a useful AI system is about controlling **what goes into the context window**: **prompt engineering** sets the instructions, **RAG** supplies fresh and private knowledge, **fine-tuning** (cheaply, with **LoRA**) changes behaviour and style, **deployment** makes it fast, reliable and affordable, and **safety** work keeps it trustworthy.

Era	Milestone	Limitation that motivated the next step
2017–2019	Transformer; GPT and BERT pre-training	Each task still needed its own fine-tuned model
2020	GPT-3 in-context learning; RAG paper (Lewis et al.)	Models were not reliable at following instructions
2021–2022	LoRA; instruction tuning and RLHF (InstructGPT); chat assistants	Full fine-tuning and serving were expensive
2023	Open-weight models; QLoRA, DPO; high-throughput serving (PagedAttention)	Hallucination, safety, evaluation at scale
2024 →	Tool use and agents, long context, multimodal and reasoning models	Reliability, security of agents, governance

1 LLM Foundations

A **large language model (LLM)** is a decoder-only Transformer (Module 03) with billions of parameters, trained to model the probability of text one token at a time. Everything it does, from answering questions to writing code, comes from repeatedly predicting the next token:

$$P(x_1, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_{<t})$$

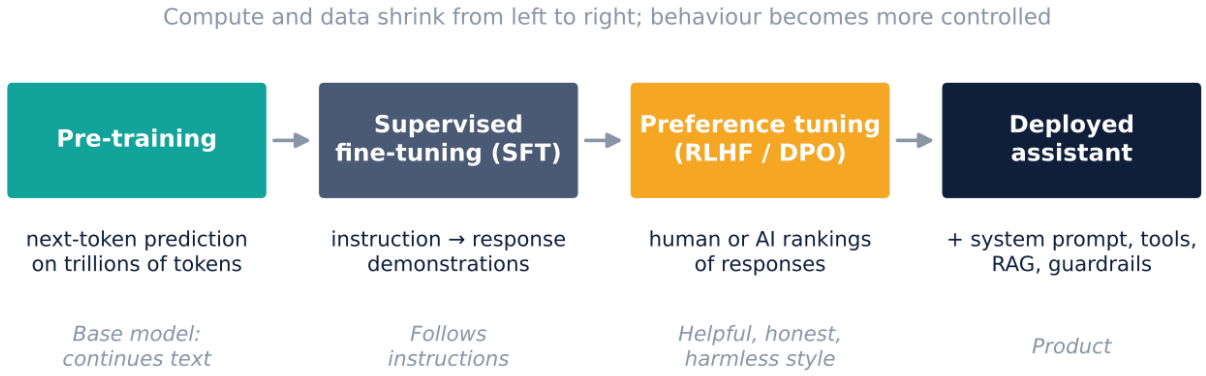


Figure 1: From base model to assistant. Pre-training gives knowledge; SFT and preference tuning give behaviour.

Stage	Data	What it teaches
Pre-training	Trillions of tokens of web, books, code	Language, world knowledge, reasoning patterns
Supervised fine-tuning (SFT)	Tens of thousands of instruction → response pairs	Follow instructions in a chat format
Preference tuning	Pairs of responses ranked by humans or AI	Prefer helpful, honest, harmless answers
System prompt + tools	Written at deployment time	Product-specific role, rules and capabilities

1.1 Tokens, context and cost

- The **context window** is the maximum number of tokens the model can attend to at once: system prompt + conversation + retrieved documents + the output it is generating. Anything outside it is invisible.
- Providers bill **per token**, usually with output tokens priced higher than input tokens because they are generated one at a time.
- English averages roughly 1.3 tokens per word; code, numbers and many non-English languages use more.

$$\text{cost} = \frac{N_{in}}{10^6}\, p_{in} + \frac{N_{out}}{10^6}\, p_{out}$$

WORKED EXAMPLE
Example prices: $3 per million input tokens, $15 per million output tokens. One request with 1,500 input and 500 output tokens costs 0.0045 + 0.0075 = **$0.012**. At 100,000 requests per day: **$1,200 per day**. Halving the prompt saves $225/day; halving the output saves $375/day.
Context budget with an 8,192-token window: system prompt 500 + 5 retrieved chunks × 800 = 4,000 + question 200 = 4,700 tokens, leaving **3,492** tokens for the answer.

1.2 Strengths and limitations

- **Strengths:** general-purpose language understanding, generation, summarization, translation, coding, extraction, and learning a new task from a few examples in the prompt (in-context learning).
- **Hallucination:** fluent but false statements, because the model is trained to produce plausible text, not to check facts.
- **Knowledge cutoff:** the weights know nothing after the training data ends, and nothing about your private data.
- **Non-determinism and prompt sensitivity:** sampling and small wording changes can change the answer.

COMMON PITFALL
Treating the model as a database. An LLM stores compressed patterns, not records; for facts that must be current, exact or private, retrieve them (Section 3) or call a tool rather than relying on the weights.

2 Prompt Engineering

Prompt engineering is designing the input so that the model reliably produces the output you need. It is the cheapest and fastest lever: no training, instant iteration. A good prompt reads like a clear brief to a capable new colleague who knows nothing about your situation.

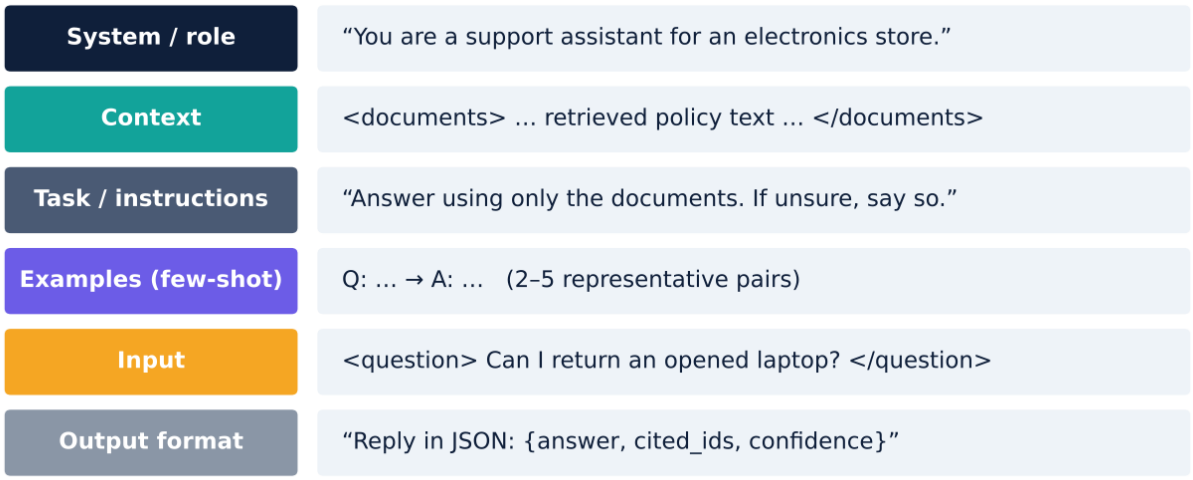


Figure 2: Anatomy of a well-structured prompt. Delimiters (XML-style tags) separate instructions from data.

2.1 Core principles

- **Be specific:** state the goal, audience, length, tone and what a good answer looks like.
- **Give context and reasons:** explaining *why* a rule exists helps the model generalize it correctly.
- **Separate instructions from data** with clear delimiters, e.g. <document> … </document>.
- **Say what to do**, not only what to avoid; positive instructions are followed more reliably.
- **Show the format** with an example or a schema, and allow “I don’t know” to reduce guessing.
- **Iterate with test cases:** keep a small set of inputs with expected outputs and re-run them after every change.

2.2 Prompting techniques

Technique	How it works	Best for
Zero-shot	Instruction only, no examples	Simple, well-known tasks
Few-shot	2–5 input → output examples in the prompt	Specific formats, labels, style
Chain-of-thought (CoT)	Ask the model to reason step by step before answering	Maths, logic, multi-step problems
Self-consistency	Sample several reasoning paths, take the majority answer	Higher accuracy on reasoning, at higher cost
Role / system prompt	Set persona, rules and scope for the whole conversation	Assistants and products
Structured output	Require JSON matching a schema; validate it	Feeding results into code
Prompt chaining	Split a task into steps, each with its own prompt	Long or complex workflows
ReAct / tool use	Interleave reasoning with tool calls and observations	Search, calculators, APIs, agents

WORKED EXAMPLE
Weak: “Summarize this.” **Strong:** “Summarize the <report> below for a non-technical manager in 3 bullet points of at most 20 words each. Focus on decisions and deadlines. If the report has no deadline, say ‘No deadline stated’.” The strong version fixes audience, length, focus and an edge case.
Self-consistency: 5 sampled answers are 42, 42, 38, 42, 40 → majority vote returns **42** (3 of 5).

2.3 Sampling parameters

Parameter	Effect	Typical setting
Temperature	Scales logits; low = focused and repeatable, high = varied	0–0.3 extraction/code; 0.7–1.0 creative writing
Top-p	Sample only from the smallest set covering probability p	0.9–1.0
Max tokens	Hard cap on output length (and cost)	Just above the longest expected answer
Stop sequences	End generation when a string appears	Closing tag, “\n\nUser:”

2.4 Tool use and agents

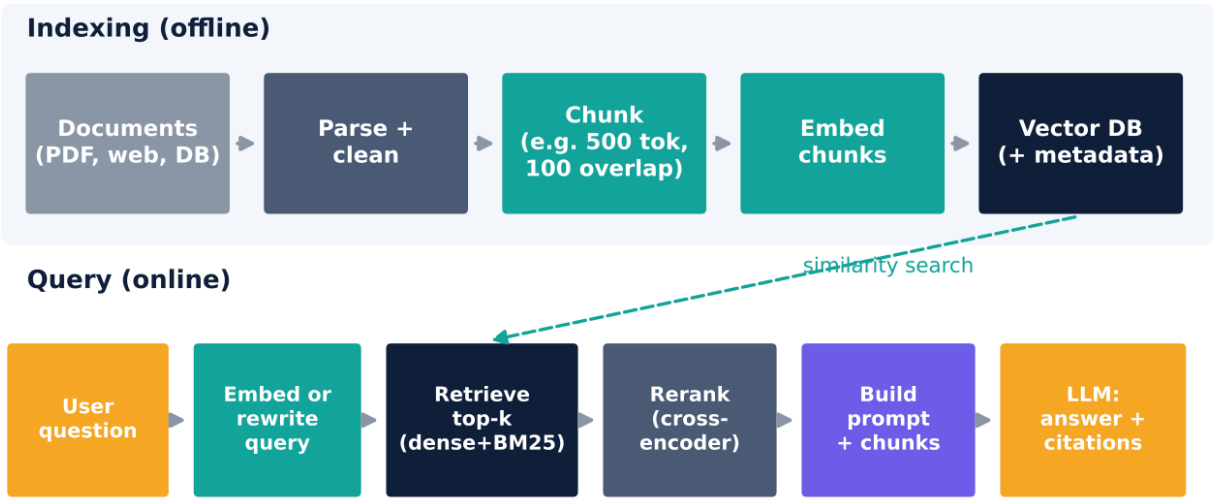
With **tool use** (function calling), the developer describes tools with names and JSON schemas. The model replies with a structured call, the application executes it and returns the result, and the model continues. An **agent** runs this loop repeatedly, choosing tools until the goal is met.

- Keep tools small, well-named and documented; validate every argument before executing.
- Give agents **least privilege** and require human confirmation for irreversible actions (payments, deletes, emails).
- Set limits on steps, time and spend, and log every tool call for debugging.

COMMON PITFALL
Prompt injection: text inside retrieved documents, web pages or emails can contain instructions such as “ignore previous instructions”. Treat all such content as **data**, never as commands (Section 6).

3 Retrieval-Augmented Generation (RAG)

RAG retrieves relevant passages from your own documents at question time and puts them in the prompt, so the model answers from evidence. It addresses the knowledge cutoff and private-data problems, reduces hallucination, enables **citations**, and updates instantly when documents change, without retraining.



Retrieval quality caps answer quality: the LLM cannot cite what was never retrieved

Figure 3: The two halves of RAG. Indexing runs once per document; the query path runs for every question.

3.1 Chunking

Documents are split into **chunks** small enough to be specific but large enough to keep meaning. Overlap stops a sentence that crosses a boundary from being lost.

$$n_{chunks} = \left\lceil \frac{N - C}{C - O} \right\rceil + 1$$

WORKED EXAMPLE
A 10,000-token document, chunk size C = 500, overlap O = 100 → stride 400. n = ⌈(10,000 − 500)/400⌉ + 1 = ⌈23.75⌉ + 1 = **25** chunks. Overlap costs storage: 25 × 500 = 12,500 tokens stored for 10,000 tokens of text.

Strategy	How	Trade-off
Fixed-size	Every N tokens, with overlap	Simple; may cut mid-idea
Recursive / structural	Split on headings, paragraphs, then sentences	Respects document structure; good default
Semantic	Split where embedding similarity between sentences drops	Coherent chunks; slower indexing
Parent-child	Search small chunks, return their larger parent section	Precise retrieval, fuller context

3.2 Embeddings and vector search

An **embedding model** maps text to a vector so that similar meanings are close. The query is embedded the same way, and the nearest chunks are retrieved, usually by cosine similarity (Module 03).

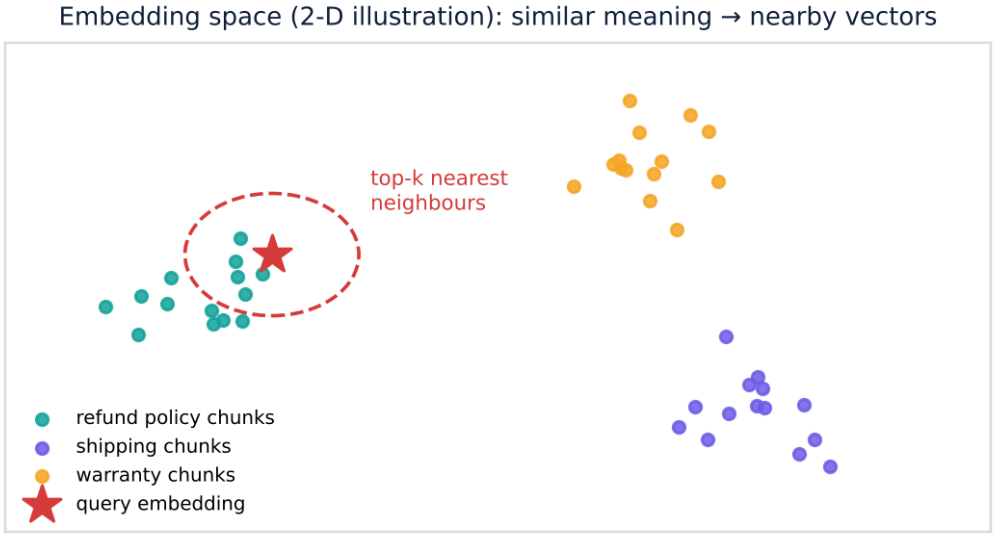


Figure 4: The query lands near the refund-policy chunks, so those are retrieved.

- Exact search compares the query with every vector; **approximate nearest neighbour (ANN)** indexes such as **HNSW** (graph-based) or **IVF** (clustered) trade a little recall for large speed-ups.
- Store **metadata** (source, date, access rights) with each vector to filter results and to cite sources.
- Use the **same embedding model** for documents and queries; changing it means re-embedding everything.

WORKED EXAMPLE
1,000,000 chunks × 768 dimensions × 4 bytes (float32) = **3.07 GB** of vectors. Storing them as 8-bit integers cuts this to about 0.77 GB at a small cost in accuracy.

3.3 Hybrid search and reranking

Dense embeddings capture meaning but can miss exact terms such as product codes or names; keyword search (**BM25**) catches them. **Hybrid search** runs both and merges the ranked lists, often with Reciprocal Rank Fusion:

$$\mathrm{RRF}(d) = \sum_i \frac{1}{k + \mathrm{rank}_i(d)}, \qquad k = 60$$

WORKED EXAMPLE
A chunk ranked 1st by BM25 and 3rd by dense search scores 1/61 + 1/63 = 0.01639 + 0.01587 = **0.0323**. A chunk ranked 2nd by dense search only scores 1/62 = 0.0161, so agreement between methods wins.

A **reranker** (cross-encoder) then reads the query and each candidate together and re-scores, e.g. retrieve the top 50 cheaply, rerank, keep the best 5. It is slower per pair but much more accurate.

3.4 Evaluating RAG

$$\mathrm{Recall@}k = \frac{\text{relevant in top } k}{\text{all relevant}}, \qquad \mathrm{MRR} = \frac{1}{|Q|}\sum_q \frac{1}{\mathrm{rank}_q}$$

WORKED EXAMPLE
Three test questions: the first relevant chunk appears at rank 1, rank 3, and not at all. MRR = (1 + 1/3 + 0) / 3 = **0.444**. Recall@5 for the third question is 0, which no prompt can fix.

Layer	Metric	Question it answers
Retrieval	Recall@k, precision@k, MRR	Did we fetch the right chunks?
Generation	Faithfulness (groundedness)	Is every claim supported by the retrieved context?
Generation	Answer relevance	Does the answer address the question?
End to end	Correctness vs reference, citation accuracy, latency, cost	Is the system good enough to ship?

3.5 Failure modes and fixes

Symptom	Likely cause	Fix
Right document exists but is not retrieved	Poor chunking, vocabulary mismatch	Hybrid search, query rewriting, better chunks
Retrieved but answer ignores it	Too many or noisy chunks; buried in the middle	Rerank, fewer chunks, put the best first
Confident wrong answer	No relevant chunk, model guesses	Instruct “answer only from context, else say unknown”
Outdated answer	Stale index	Incremental re-indexing with timestamps
User sees data they should not	No access control at retrieval	Filter by user permissions in metadata

Advanced patterns include **query rewriting** and **HyDE** (embed a hypothetical answer instead of the question), **multi-hop** retrieval for questions that need several documents, and **agentic RAG**, where the model decides when and what to search.

4 Fine-Tuning and LoRA

Fine-tuning continues training a pre-trained model on your examples, changing its weights. It is the right tool for **behaviour** (format, style, tone, a narrow skill, a smaller faster model), not for adding facts that change often.

Need	Prompting	RAG	Fine-tuning
Up-to-date or private facts	Weak	**Best**	Weak (stale, may hallucinate)
Consistent format or style	Good	No effect	**Best**
New narrow skill or domain language	Limited	Limited	**Best**
Citations and traceability	No	**Best**	No
Cost and time to start	**Lowest**	Medium	Highest (data + training)

KEY IDEA
Try in this order: **prompting** → **RAG** for knowledge → **fine-tuning** for behaviour. They combine well: a fine-tuned model can still use RAG.

4.1 Why full fine-tuning is expensive

Full fine-tuning updates every parameter. With the Adam optimizer in mixed precision each parameter needs about **16 bytes**: 2 (FP16 weight) + 2 (gradient) + 4 (FP32 master copy) + 8 (two Adam moments).

WORKED EXAMPLE
7B parameters × 16 bytes = **112 GB** before activations: several data-centre GPUs. Each fine-tuned copy is also a full 14 GB checkpoint (FP16) to store and serve.

4.2 LoRA: Low-Rank Adaptation

LoRA (Hu et al., 2021) freezes the pre-trained weight W_0 and learns a low-rank update $\Delta W = BA$, with rank r much smaller than the matrix dimensions. The idea is that the change needed to adapt a model lies in a low-dimensional subspace.

$$h = W_0 x + \frac{\alpha}{r} BA x, \qquad B \in \mathbb{R}^{d \times r},\ A \in \mathbb{R}^{r \times k}, \qquad \text{params} = r(d + k)$$

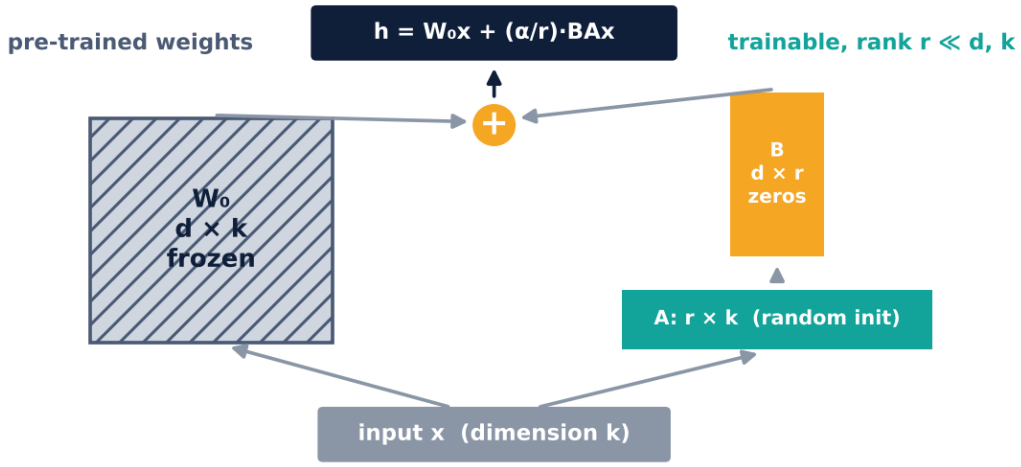


Figure 5: LoRA adds a trainable low-rank path beside each frozen weight matrix.

- **B starts at zero**, so at step 0 the model behaves exactly like the original.
- **α / r** scales the update; α is often set to r or 2r.
- After training, BA can be **merged** into W_0, so inference has no extra latency; or kept separate so many small adapters share one base model.

WORKED EXAMPLE
Projection matrix 4096 × 4096, rank r = 16: LoRA trains 16 × (4096 + 4096) = **131,072** parameters instead of 16,777,216 (0.78%).
Applying it to the query and value projections of all 32 layers: 2 × 32 × 131,072 = **8,388,608 ≈ 8.4M** trainable parameters, about **0.12%** of a 7B model. The adapter file is roughly 17 MB in FP16.

Hyper-parameter	Typical range	Effect
Rank r	4–64	Capacity of the update; higher r = more parameters
Alpha α	r to 2r	Strength of the update
Target modules	Attention (q, k, v, o), often MLP too	More modules = more capacity, more memory
Learning rate	≈ 1e-4 to 2e-4	Higher than full fine-tuning because few weights move
Epochs	1–3	More risks overfitting small datasets

4.3 QLoRA

QLoRA (Dettmers et al., 2023) loads the frozen base model in **4-bit** (NF4 data type, with double quantization of the scaling constants) and trains LoRA adapters in 16-bit on top. Paged optimizers absorb memory spikes. A 7B model's weights shrink from 14 GB to about **3.5 GB**, so fine-tuning fits on a single consumer GPU, with quality close to 16-bit LoRA.

4.4 Preference tuning

$$\max_{\pi} \; \mathbb{E}_{x,\, y \sim \pi}[r(x, y)] - \beta \, \mathrm{KL}(\pi \,\|\, \pi_{ref})$$

- **RLHF:** collect human rankings of responses → train a **reward model** r(x, y) → optimize the policy π with reinforcement learning (e.g. PPO). The KL term keeps π close to the reference model so it does not exploit the reward model.
- **DPO** (Direct Preference Optimization) skips the reward model and RL loop, training directly on (chosen, rejected) pairs with a classification-style loss: simpler and more stable.
- **Constitutional / AI feedback:** a written set of principles guides an AI to critique and rank responses, reducing the amount of human labelling.

4.5 Data and evaluation

- **Quality beats quantity:** a few hundred to a few thousand clean, consistent, diverse examples often beat a large noisy set.
- Match the **chat template** the base model expects, and train only on the response tokens.
- Hold out a test set; compare against the base model plus a good prompt, which is the baseline to beat.
- Watch for **catastrophic forgetting**: check general abilities, not only the target task.

COMMON PITFALL
Fine-tuning to "teach facts" usually disappoints: the model learns the *style* of the answers and can confidently invent details. Use RAG for knowledge and fine-tuning for behaviour.

5 Production Deployment

A demo becomes a product when it is **fast, reliable, affordable, observable and secure** under real traffic. Most of that work is engineering around the model call.

A production LLM system is mostly software engineering around one model call

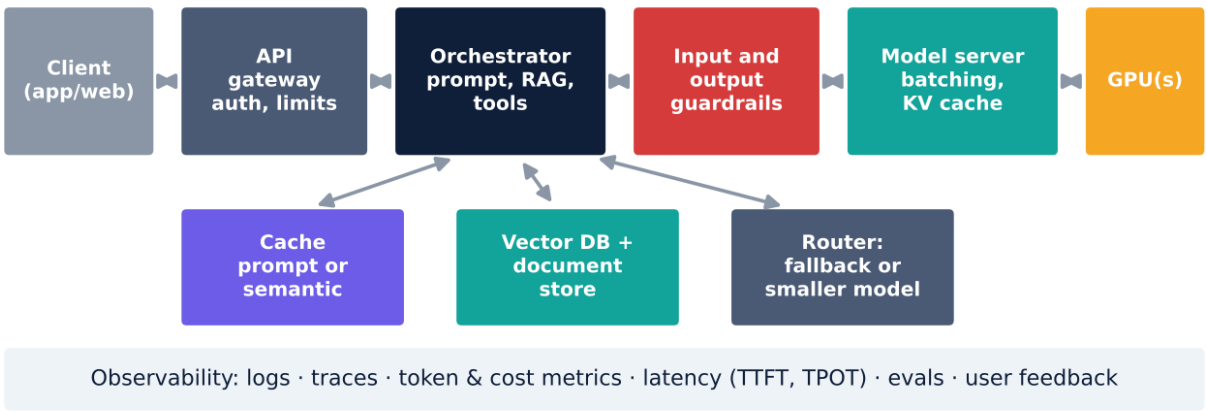


Figure 6: Reference architecture for an LLM application.

5.1 GPU memory

$$M_{weights} = N_{params} \times b, \qquad M_{KV} = 2 \cdot L \cdot d_{model} \cdot n_{ctx} \cdot b \ \text{(per sequence)}$$

WORKED EXAMPLE
Weights: 7B parameters × 2 bytes (FP16) = **14 GB**.
KV cache for L = 32 layers, d_{model} = 4096, a 4,096-token sequence in FP16: 2 × 32 × 4096 × 4096 × 2 = 2,147,483,648 bytes = **2 GiB per sequence**. Ten concurrent long conversations need 20 GiB more than the weights. Grouped-query attention with 4× fewer key/value heads cuts this to 0.5 GiB.

5.2 Quantization

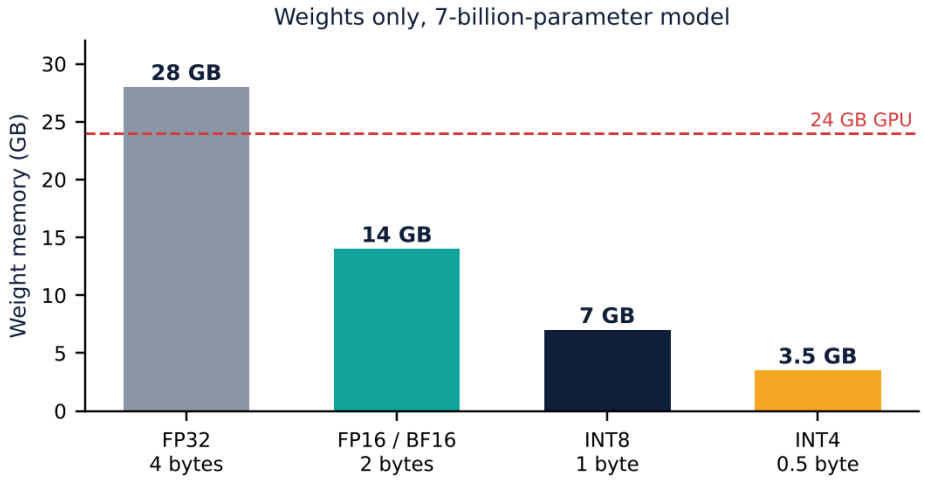


Figure 7: Lower precision shrinks memory linearly. INT4 lets a 7B model fit comfortably on a 24 GB GPU.

- **Post-training quantization** (e.g. GPTQ, AWQ, GGUF formats) converts trained weights to 8 or 4 bits with small accuracy loss; always re-run your evaluation set afterwards.
- Smaller weights also mean **faster decoding**, which is usually limited by memory bandwidth, not arithmetic.
- **Distillation** trains a small model to imitate a large one for a specific task: cheaper and faster to serve.

5.3 Serving and latency

$$T_{total} = \text{TTFT} + N_{out} \times \text{TPOT}$$

WORKED EXAMPLE
Time to first token (TTFT) 0.4 s, 300 output tokens at a time per output token (TPOT) of 25 ms: 0.4 + 300 × 0.025 = **7.9 s**. Streaming shows text after 0.4 s, so the app feels fast even though the full answer takes 7.9 s. Asking for a 100-token answer would cut the total to 2.9 s.

Technique	What it does	Gain
KV cache	Reuses keys/values of earlier tokens	Each step computes only the new token
Continuous batching	Adds and removes requests from a batch at every step	Much higher GPU utilization and throughput
PagedAttention	Stores KV cache in fixed-size pages, like virtual memory	Less fragmentation, more concurrent users
Prefix / prompt caching	Reuses computation for a repeated prompt prefix	Lower TTFT and cost for long system prompts

Technique	What it does	Gain
Speculative decoding	Small draft model proposes tokens; large model verifies several at once	Lower latency, same output distribution
Streaming	Sends tokens as they are generated	Better perceived latency

5.4 Cost control

- **Model routing:** send easy requests to a small, cheap model and hard ones to a large model.
- **Caching:** exact-match or **semantic** caches (embedding similarity) answer repeated questions instantly; a 30% hit rate removes about 30% of model calls.
- Trim prompts, limit retrieved chunks, set sensible max-token limits and use batch APIs for offline jobs.

5.5 Reliability

$$t_n = \min\left(t_{max},\ t_0 \cdot 2^{n}\right) + \text{jitter}$$

WORKED EXAMPLE
Exponential backoff with t_0 = 1 s: retries wait about 1, 2, 4, 8 s, plus a random jitter so thousands of clients do not retry at the same instant. Retry only transient errors (timeouts, rate limits, overloads), never invalid requests.

- Set timeouts, fall back to another model or a cached answer, and degrade gracefully.
- Validate structured output against its schema and retry once with the error message if it fails.
- Respect provider **rate limits** with a queue, and protect your own API with per-user limits.

5.6 LLMOps: observability and evaluation

- Log prompts, retrieved context, outputs, tool calls, token counts, latency and cost, with PII handling.
- Maintain an **evaluation set** and run it in CI before every prompt, model or retrieval change (regression tests).
- Combine automatic metrics, **LLM-as-judge** scoring (checked against human labels) and human review.
- Version prompts like code; roll out changes with A/B tests or canaries and watch user feedback and drift.

COMMON PITFALL
Shipping a prompt change without re-running the evaluation set. Small edits can fix one case and silently break ten others; LLM behaviour is not covered by ordinary unit tests.

6 AI Safety and Ethics

AI systems can cause harm through error, misuse or attack. Safety and ethics work identifies these risks, reduces them with technical and organizational controls, and makes responsibility clear.

Risk	Example	Main mitigations
Hallucination	Invented citation or policy	RAG with citations, “say unknown”, grounding checks
Prompt injection	Web page tells an agent to email user data	Treat retrieved content as data, least privilege, confirmations
Jailbreaks	Role-play tricks to bypass refusals	Alignment training, input/output classifiers, red teaming
Bias and unfairness	Different loan advice by group	Diverse data, fairness testing, human oversight

Risk	Example	Main mitigations
Privacy leakage	Model repeats personal data from training or context	Data minimization, PII redaction, access control
Toxic or harmful content	Harassment, dangerous instructions	Content policies, moderation, refusals
Misuse	Spam, fraud, impersonation at scale	Usage policies, monitoring, rate limits
Over-reliance	Users accept wrong medical or legal output	Clear limitations, citations, human review for high stakes
Intellectual property	Reproducing copyrighted text or code	Filters, attribution, licensed data

6.1 Prompt injection

- **Direct injection:** the user types instructions meant to override the system prompt.
- **Indirect injection:** instructions hidden in content the model reads (documents, web pages, emails, tool results). Most dangerous for agents with tools, because the attacker never talks to the model directly.
- No prompt wording fully prevents injection; design so that a fooled model **cannot do much harm**: limited tools, no secrets in the prompt, human approval for side effects, and output checks.

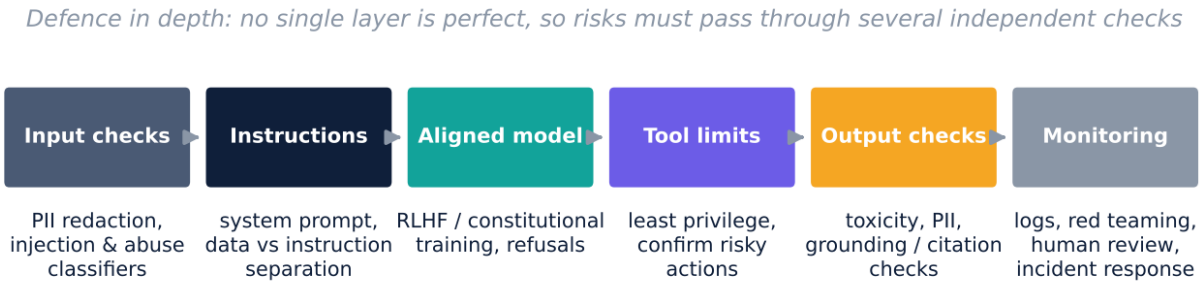


Figure 8: Defence in depth for an LLM application.

6.2 Alignment, red teaming and evaluation

- **Alignment** training (RLHF, DPO, constitutional methods) teaches models to be helpful, honest and harmless and to refuse clearly harmful requests without over-refusing benign ones.
- **Red teaming:** people and automated attackers deliberately try to elicit harmful or incorrect behaviour before and after launch.
- **Safety evaluations** measure refusal rates, jailbreak robustness, bias and hallucination on fixed test sets, tracked across versions.

6.3 Fairness

$$\text{DPD} = \left|P(\hat{y}=1 \mid A) - P(\hat{y}=1 \mid B)\right|, \qquad \text{DI} = \frac{P(\hat{y}=1 \mid B)}{P(\hat{y}=1 \mid A)}$$

WORKED EXAMPLE
An AI screening tool approves 60% of group A and 45% of group B. Demographic parity difference = 0.60 − 0.45 = **0.15**. Disparate impact ratio = 0.45 / 0.60 = **0.75**, below the common “four-fifths” (0.8) rule of thumb, so the system should be investigated. Different fairness definitions can conflict; choose them with domain experts.

6.4 Privacy and governance

- **Privacy:** collect the minimum data, redact PII before logging or sending to third parties, set retention limits, and enforce document-level permissions in RAG.
- **Transparency:** tell users they are interacting with AI; publish **model cards** and dataset documentation.
- **Human oversight:** keep a human in the loop for high-stakes decisions (health, hiring, credit, legal).
- **Frameworks and law:** the NIST AI Risk Management Framework (Govern, Map, Measure, Manage) and the EU AI Act's risk-based tiers (unacceptable, high, limited, minimal) set expectations for documentation, testing and oversight.
- **Accountability:** named owners, incident response plans and audit logs.

COMMON PITFALL
Safety is not a final checklist. Risks change with new features (adding a tool or a data source can create a new injection path), so threat-model and re-test whenever the system's capabilities change.

7 Revision Kit

7.1 Choosing the right lever

Situation	First choice	Why
Answer questions about company documents	RAG	Fresh, private knowledge with citations
Always output a strict JSON schema	Structured output prompt; then fine-tune if needed	Format is behaviour
Domain tone and terminology everywhere	LoRA fine-tune	Consistent style without long prompts
Multi-step task needing calculations or APIs	Tool use / agent	Exact results from real systems
Too slow or too expensive	Smaller model, quantization, caching, routing	Serving efficiency
High-stakes decisions	Human in the loop + evaluation + logging	Accountability and safety

7.2 One-page formula sheet

Concept	Formula / rule
Autoregressive LM	$P(x) = \Pi_t P(x_t \mid x_{<t})$
Token cost	$(N_{in}/10^6)\cdot p_{in} + (N_{out}/10^6)\cdot p_{out}$
Context budget	system + history + retrieved + question + output ≤ window
Number of chunks	$\lceil (N - C) / (C - O) \rceil + 1$
Vector storage	chunks × dimensions × bytes per value
Reciprocal Rank Fusion	$\Sigma_i\ 1 / (k + \text{rank}_i(d))$, $k = 60$
MRR	mean of 1 / rank of the first relevant result
LoRA forward	$h = W_0x + (\alpha/r)\cdot BAx$
LoRA parameters	$r(d + k)$ per adapted matrix
Full fine-tune memory	≈ 16 bytes per parameter (Adam, mixed precision)
Weight memory	parameters × bytes (FP16 = 2, INT8 = 1, INT4 = 0.5)
KV cache	$2 \cdot L \cdot d_{model} \cdot n_{ctx}$ · bytes per sequence
Latency	$TTFT + N_{out} \times TPOT$
Backoff	$t_n = t_0 \cdot 2^n$ + jitter
Fairness	$DPD = \lvert P_A - P_B \rvert$; $DI = P_B / P_A$ (flag if < 0.8)

7.3 Classic mistakes

- Using fine-tuning to add frequently changing facts instead of RAG.
- Vague prompts with no format, audience or examples, then blaming the model.
- Mixing instructions and untrusted data without delimiters; trusting retrieved text as commands.
- Judging RAG only by the final answer without measuring retrieval recall.
- Stuffing the context with dozens of chunks: more noise, more cost, worse answers.
- Changing the embedding model without re-indexing the documents.
- Forgetting the KV cache when sizing GPU memory for many concurrent users.
- Quantizing a model and shipping it without re-running evaluations.
- Retrying non-transient errors, or retrying without backoff and jitter.
- Giving an agent broad permissions and no human confirmation for irreversible actions.
- Treating fluent output as verified fact in high-stakes settings.
- Shipping prompt changes without a regression evaluation set.

7.4 Self-check before the MCQ assessment

- Can you estimate the cost of a request and the remaining context budget?
- Can you rewrite a vague prompt into a specific, structured one and pick a prompting technique?
- Can you count chunks with overlap, compute RRF and MRR, and name a fix for each RAG failure mode?
- Can you compute LoRA parameters and explain why B is initialized to zero?
- Can you compute weight memory, KV-cache size and end-to-end latency?
- Can you explain indirect prompt injection and compute a disparate impact ratio?

KEY IDEA

Next step: attempt the **Module 05 MCQ Assessment**. For each miss, return to the matching section, redo the worked example by hand and write one sentence on why the wrong option was tempting.